PETALS | Page No.
Date / /

THEORY OF COMPUTATION

It is a branch of CS that deals with how efficiently a problem can be solved on a computational model by using an Algorithm. I.e. Theory of computation is concerned with how problems can be efficiently solved using Algorithms.

Fundamental concepts of TOC :-

1) Complexity Theory → obj :- to classify problems as
2) Computability Theory → easy ones & hard ones.
3) Automata Theory → obj :- to classify problems as solvable & not-solvable.

↳ obj :- It is a branch of theoretical CS/CS & maths that studies abstract machines (Automata) & the types of problem they can solve.

It provides the foundation for designing & Analyzing computer systems, compilers, text processing & even Artificial Intelligence (AI)

Q what is Automata?

A An Automaton is a mathematical model of computation it takes an input, changes its states acc. to transition rules & produces an output accept / reject or some computed result.

Types of Automata :-

1. Finite Automata → • It is simplest model of computation
  • It has a finite set of states
  • It is used to recognize regular languages
  • It is of 2 types
    i) DFA - Deterministic
    ii) NFA - Non-Deterministic.

2. Push-Down Automata (PDA): • It has a stack for additional memory
• It recognizes context-free languages.
• It provides the basis for parsing in compilers

Not in Syllabus → 3. Linear-Bounded Automata (LBA): It is like a turning machine but with limited tape length. (context-sensitive language)

4. Turing Machine → It is the most powerful model of computation. It has infinite tape memory. It recognizes recursively & enumerable languages.

HW 1) Set
2) Relation & Function
3) Sequence & Tuple
4) Tree
5) Graph.

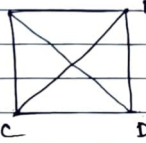
1) Set → Set builder method, Roster method. repetition not allowed.

$A = \{x \mid x \text{ is even \& less than equal to } 20\}$.
$A = \{2, 4, 6, 8 - \text{—} 20\}$.

2) Sequence:
e.g. $X = (2, 4, 9, 15)$ repetition is allowed,
$X \neq Y$ $Y = (15, 9, 4, 2)$

3) Tuple: It defines about no. of elements present in a sequence.
$Y = (15, 9, 4, 2, 2)$ 5-tuple.

Graph
$G = (V, E)$



→ complete Graph
→ 3-regular Graph

$V = \{A, B, C, D\}$
$E = \{AB, BC, CD, AD, AC, BD\}$

Proof by construction

Theorem For each even number n greater than 2, ∃ a 3-regular graph with n-vertices

Pf: consider the set of vertices as $V = \{1, 2, - \text{—} n\}$
& set of edge as $E = \{(i, i+1) \mid i = 1, 2, - \text{—} n-1\} \cup \{(n, 1)\} \cup$

$E = \{(i, i+1) \mid i = (1, 2, \cdots n-1)\} \cup \{n, 1\} \cup$
$\{(i, i+n/2) \mid i = (1, 2, \cdots n/2)\}$.

H/W
1) using Pf by contradiction show that $\sqrt{2}$ is irrational
2) using Pf by induction show that
$2^{2n} \neq O(2^n)$, $\forall n$

PETALS | Page No.
Date / /

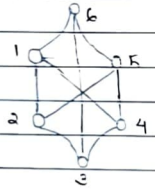
Proof by contruction

$V = \{1, 2, 3, \cdots, n\}$

$E = \{\}$

$n = 6$.

$E = \{(1,2), (2,3), (3,4), (5,6)\} \cup \{(6,1)\} \cup \{(1,4), (2,5), (3,6)\}$



Alphabet :- It is a finite set of symbols, which is denoted by Σ

Symbol: It is an individual object, which can be a character, a special character or a number.

$\{a, b, c, \cdots, y, z\}$
$\{A, B, C, \cdots, Y, Z\}$
$\{\#, @, \%, +, -, /\}$
$\{0, 1, 2, \cdots 9\}$

$\Sigma = \{a, b\}$, $\Sigma_2 = \{0, 1, \#, a, b\}$
$\Sigma_1 = \{0, 1, 2\}$

String :- It is a finite collections of symbols taken from the alphabet Σ.

Ex. let $\Sigma = \{0, 1\}$
Now "001101" is a string.

PETALS | Page No.
Date / /

$\omega = 001101$, $|\omega| = 6$.

* Empty string : A string with length zero.
i.e $|\omega| = 0$
represented by ε (Epsilon)

* Language - A language is a collection of strings over an ~~integ~~ Alphabet Σ

e.g- let $\Sigma = \{0, 1\}$.

1. Define a language L that will contain all strings of length 2 over Σ.

$L = \{01, 10, 11, 00\}$.

2. Define a language L that contain all the strings starting with 1 symbol over Σ.

$L = \{1, 10, 11, 100, 101, 110, 111, \cdots\}$

3. Find a language that will contain all the strings which contains even no. of 1's & length ≤ 4.

$L = \{0101, 0011, 1001, 1010, 1100, 0110, 1111\}$

Operations on languages :-

1) Union
2) Concatenation
3) Intersection
4) complementation
5) closure / Kleene closure / kleene star
6) positive closure

PETALS | Page No. | Date / /

Finite Automata:-

* It consists of a finite set of states & a set of transitions from state-state that occurs on input symbols choosen from an Alphabet Σ

* It can be depicted as a graph like figures with
  i) An arrow (⟶) that represents the tranition
  ii) single circle (○) that represents the single state
  iii) Double circle (◎) that represents the final state/accept state
  iv) An arrow, preceded by a single circle (from nowhere) (⟶○) that represents the initial state & final state are same at that time a single arrow is preceded by a double circle ⟶◎

TYPES

* DFA :→ (Deterministic FA is nothing but a finite
* NFA Automata whose operation is completely determined by their input.
  — It is mathematically defined as a 5-tuple
  $(Q, \Sigma, \delta, q_0, F)$ where here Q is
  1) the finite set of states
  2) Σ is the finite set of input symbols.
  3) δ is the transition function defined as
  $\delta : Q \times \Sigma \longrightarrow Q$.



4) initial state

$Q = \{q_0, q_1\}$
$\Sigma = \{0, 1\}$,

$\delta(q_0, 1) = q_1$,

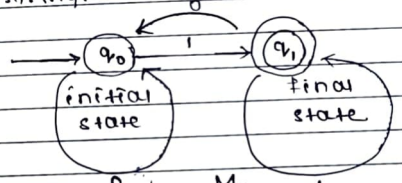
PETALS | Page No. | Date / /

5) $F \subseteq Q$ is a set of final/accept states.

** Example :

Let $\Sigma = \{0, 1\}$

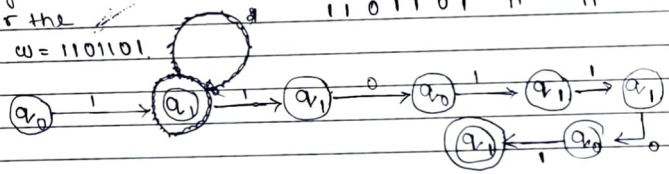
Design a DFA that will accept all strings that strings end with a 1 over Σ



⟶ DFA

① is accepted.
01 "
111 "
1101101 " "

working of DFA for the string w = 1101101



* 5-tuple form → $\{Q, \Sigma, \delta, q_0, F\}$.

$\{Q = \{q_0, q_1\}, \Sigma = \{0, 1\}, \delta, q_0, \{q_1\}\}$

(*) δ is represented by the tranition table

input / status	0	1
q_0	q_0	q_1
(q_1)	q_0	q_1

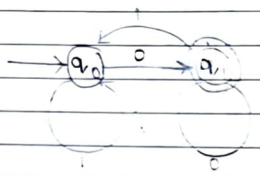
PETALS | Page No.
Date / /

* Example:

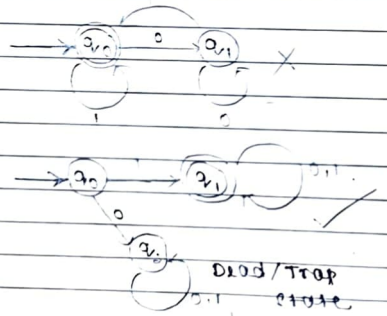
Design a DFA that will accept all strings ending with symbol over the alphabet $\Sigma = \{0, 1\}$.

$\{Q = \{q_0, q_1\}, \Sigma = \{0, 1\}, \delta; q_0, q_1\}$

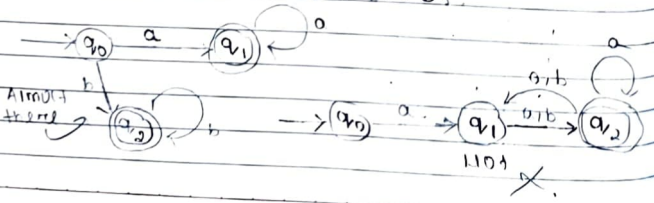
1101010

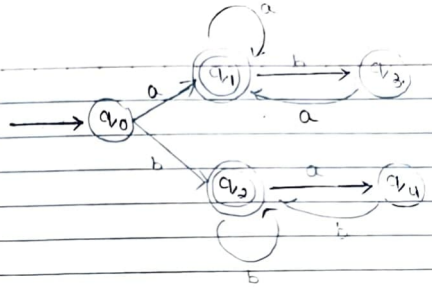


* which starts at 1.



* Starts & ends at symbol 'a' or same at symbol 'b' over the alphabet $\Sigma = \{a, b\}$.





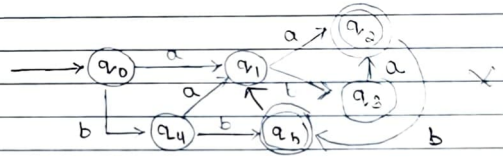
Transition Table ??

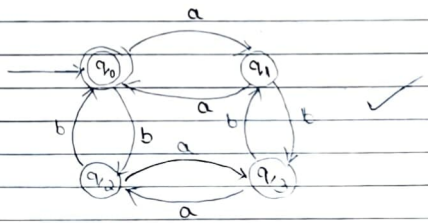
Design of DFA:

* construct a DFA to accept all the strings over the Alphabet $\Sigma = \{a, b\}$ such that the string must contain even no. of a's & even no of b's.

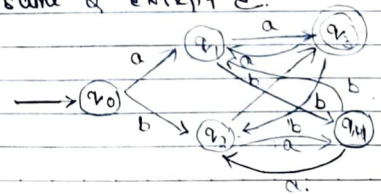
ϵ, aa, bb, aabb, bbaa, abab, baba, baab, abba, aabaab.

Trial





Ex6 same Q except ϵ.

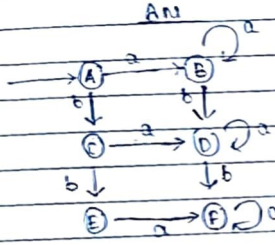


PETALS | Page No.
Date / /

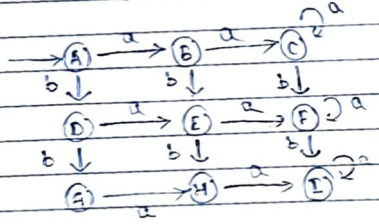
atleast one 'a' & exactly 2 b's.

Ex-10 construct a DFA that accept all strings over $\Sigma = \{a, b\}$ that contains atleast 2 a's & exactly 2 b's

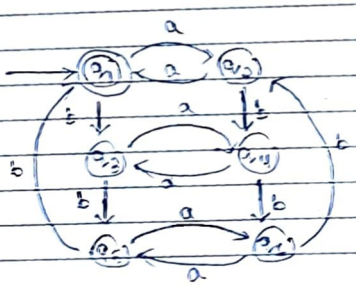
Ans



Ex-11

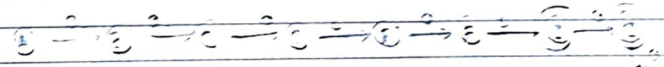


Ex-12



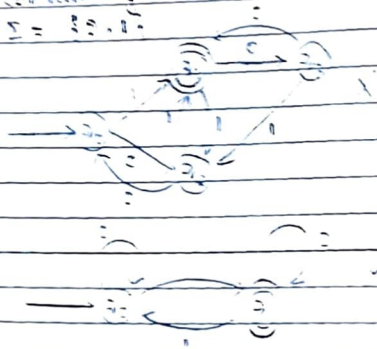
Transition:

i) $w_1 = aaabaaba$

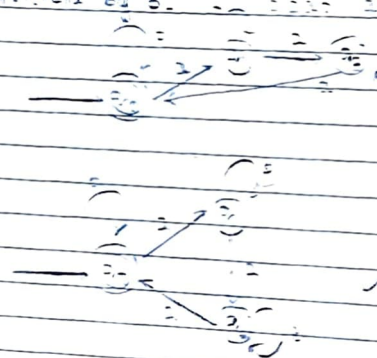


PETALS | Page No.
Date / /

Ex-9 - Design a DFA that accepts all strings over $\Sigma = \{a, b\}$ that contains atleast one 'a' & exactly two b's.

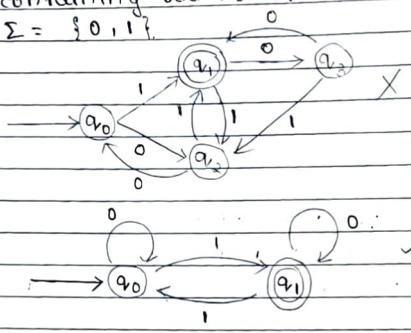


Ex-8 - Design a DFA that accepts all strings over $\Sigma = \{a, b\}$ that contains exactly two b's & any number of a's.

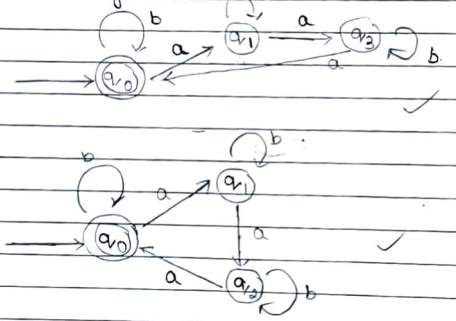


Ex-7 - Design a DFA that accepts all strings over $\Sigma = \{a, b\}$ that contains atleast two b's & any number of a's.

Ex-7 – Design a DFA that accepts all strings containing odd no. of 1's over the alphabet $\Sigma = \{0, 1\}$.



Ex-8 :- Design a DFA that accepts all strings over $\Sigma = \{a, b\}$ s.t the no. of a's in the string is divisible by 3.

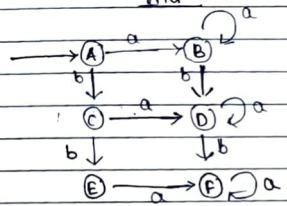


Ex-9 : construct a DFA that accepts all strings over $\Sigma = \{a, b\}$, in which the number of a's divisible by 2 & no. of b's divisible by 3.

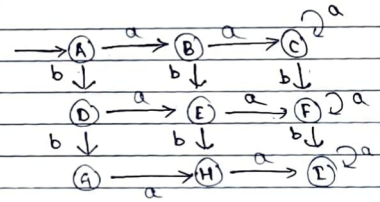
Ex-10 : construct a DFA that accept all strings over $\Sigma = \{a, b\}$ that contains atleast 2 a's & exactly 2 b's.

AN

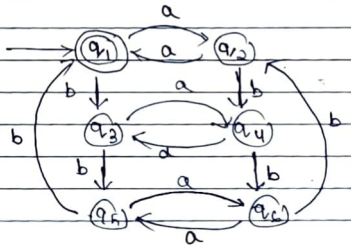
Ex-10



Ex-11

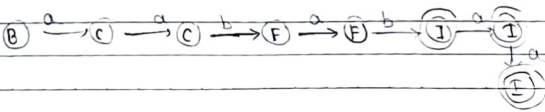


Ex-9

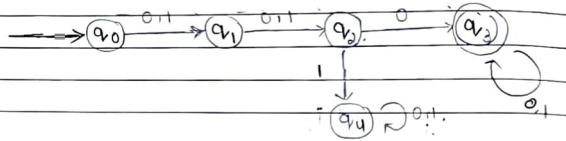


Transition:

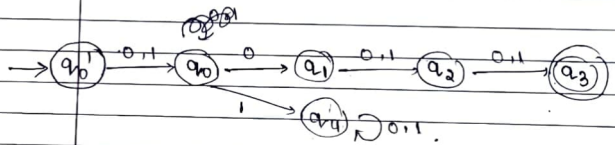
1) w_1 = aaaba baa



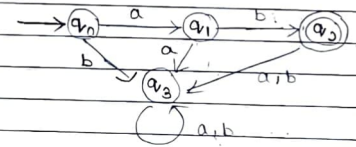
Ex-12 Design a DFA that will accept all strings of length $x = 3$ and the 3rd symbol from the of the string from the beginning is 0. over the alphabet $\Sigma = \{0, 1\}$.

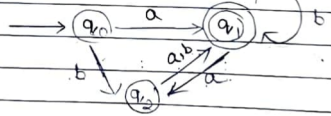


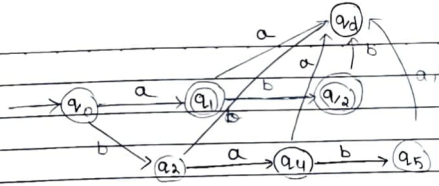
13 From the last.



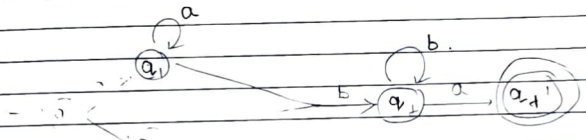
* {ab}.





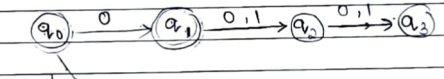


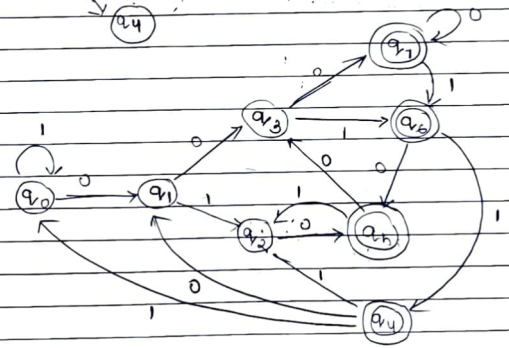
end with 'ba'.



$L = \{ba, aba, bba, \text{------}\}$?

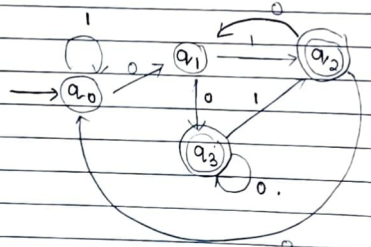
Ex-12



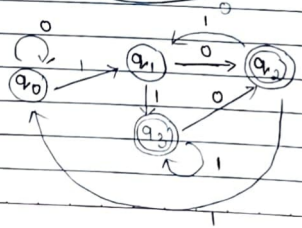


state \ inputs	1	0
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_5
q_3	q_6	q_7
q_4	q_0	q_1
q_5	q_2	q_3
q_6	q_4	q_5
q_7	q_6	q_7

state \ input	0	1 for 1
	1	0 for 0.
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_0	q_1
q_3	q_2	q_3



for second last position to be 0,



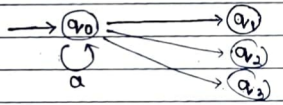
for 1

Non-Deterministic Finite Automata (NFA)

* The NFA is a finite state machine in which for some given state q an input symbol the transition function $\delta(q)$ may lead to a set of states i.e. a subset of Q.

* It means in a Non-Deterministic machine several choices may exist for the next state on any input symbol.

$\delta(q_0, a) \rightarrow q_1$ → 



$\delta(q_0, a) \rightarrow q_0, q_1, q_2, q_3$

* Every state of DFA always has exactly one exiting transition Arrow for each symbol in the Alphabet Σ, whereas when an NFA a state may have zero, one or many exiting Arrows for each Alphabet symbol

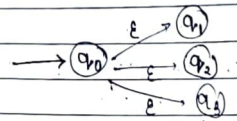


$\delta(q_0, a) \rightarrow \phi$

* In a DFA, labels on the transition arrow are symbols from the Alphabet. But NFA can have an Arrow with the label ϵ (called as ϵ-NFA) $\epsilon \notin \Sigma$

* In general an NFA may have arrows labelled with members of the Alphabet Σ or ϵ. 0, 1 or many Arrows may exit

from each state with the label ε.



$\delta(q_0, \varepsilon) = \phi$

zero transition Arrow.

Formal defⁿ of LIFA:

The LIFA is formally defined as a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where,

i) Q → finite set of states.
ii) Σ → finite set of input symbols
iii) q_0 → initial state
iv) F → subset of Q (set of final states/ Accepting states)
v) δ → it is transition function, which is defined as:

$\delta : Q \times \Sigma \rightarrow 2^{|Q|}$ → power set of Q (set of all subsets of Q)

e.g. consider the LIFA has 2 states i.e $Q = \{q_0, q_1\}$ $\Sigma = \{a, b\}$, $P = \{\phi, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}$

① → q_0
② → q_0 (a)
③ → q_0 —a→ q_1
④ → q_0 —a→ q_1 (a)

$|Q| = 2$
$2^{|Q|} = 2^2 = 4$

① $\delta(q_0, a) = \phi$ ↦ going nowhere
② $\delta(q_0, a) = q_0$
③ $\delta(q_0, a) = q_1$
④ $\delta(q_0, a) = \{q_0, q_1\}$

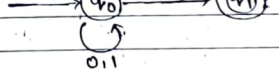
Q. consider the LIFA has 3 states i.e. $Q = \{q_0, q_1, q_2\}$ alphabet $\Sigma = \{a, b\}$. Find possible transition

$P = \{\phi, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_1, q_2\}, \{q_0, q_2\}, \{q_0, q_1, q_2\}\}$

No. of transitions $= 2^{|Q|} = 2^3 = 8$

① → q_0 $\quad \delta(q_0, a) = \phi$
② → q_0 (a) $\quad \delta(q_0, a) = q_0$
③ → q_0 —a→ q_1 $\quad \delta(q_0, a) = q_1$
④ → q_0 —a→ q_2 $\quad \delta(q_0, a) = q_2$
⑤ → q_0 —a→ q_1, q_0 —a→ q_2 $\quad \delta(q_0, a) = \{q_1, q_2\}$
⑥ → q_0 (a), q_0 —a→ q_2 $\quad \delta(q_0, a) = \{q_0, q_2\}$
⑦ → q_0 (a), q_0 —a→ q_1 $\quad \delta(q_0, a) = \{q_0, q_1\}$
⑧ → q_0 (a), q_0 —a→ q_1, q_0 —a→ q_2 $\quad \delta(q_0, a) = \{q_0, q_1, q_2\}$

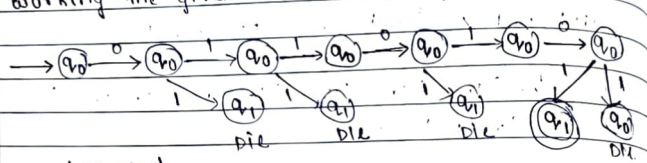
Q construct a LIFA that accepts all strings ending with the symbol 1 over the $\Sigma = \{0, 1\}$



δ table

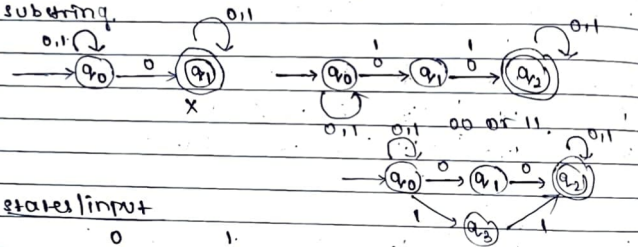
input / states	0	1
q_0	q_0	$\{q_0, q_1\}$
q_1	ϕ	ϕ

working the given NFA on a string $w = 0110101$



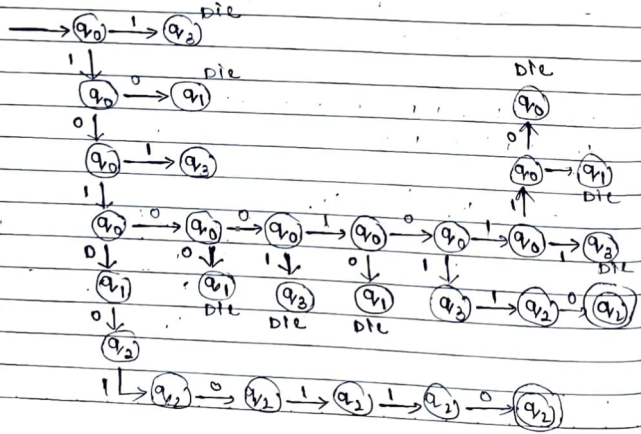
Accepted.

Q Construct a NFA that accepts all string over $\Sigma = \{0, 1\}$ in which the strings contain "00" as substring.

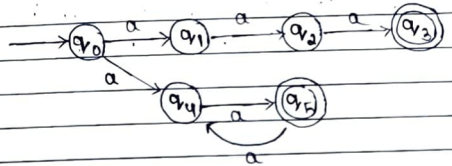


states/input	0	1
q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\{q_2\}$	ϕ
q_2	$\{q_2\}$	$\{q_2\}$

$w = 1010010110$



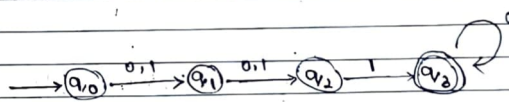
e.g Design an NFA that accepts all strings that contain exactly 3 a's or even no. of a's except 2, over the input alphabet $\Sigma = \{a\}$



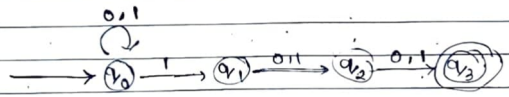
Transition Table

states \ input	a
q_0	$\{q_1, q_4\}$
q_1	q_2
q_2	q_3
q_3	ϕ
q_4	q_5
q_5	q_4

e.g Design an NFA that accepts all the strings where the 2nd symbol '1' appears in the 3rd place from the beginning (i.e. left end) of the string over input alphabet $\Sigma = \{0, 1\}$.



states/input	0	1
q_0	q_1	q_1
q_1	q_2	q_2
q_2	ϕ	q_3
q_3	q_3	q_3

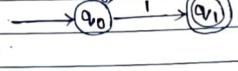


PETALS | Page No.
Date / /

Equivalence of NFAs and DFAs (i.e. conversion from NFA to DFA)

1. Create the transition table of the given NFA
2. Create the transition table for the DFA by referring the transition table of the NFA
3. Create the DFA

e.g. convert the following NFA to its equivalent DFA



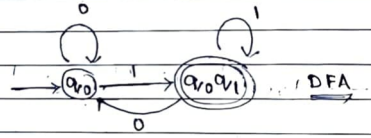
$\delta(q_0q_1, 0) = \delta(q_0, 0) \cup \delta(q_1, 0)$

NFA

	input	
states	0	1
$\to q_0$	q_0	$\{q_0, q_1\}$
q_1	ϕ	ϕ

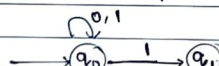
DFA

	0	1
$\to q_0$	q_0	$q_0q_1 \cong q_{01}$
q_0q_1	q_0	q_0q_1



DFA

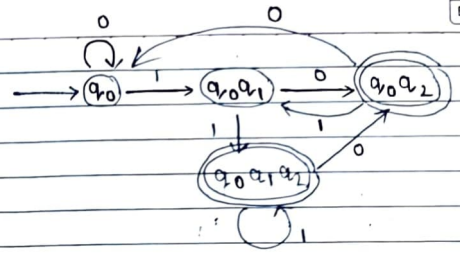
e.g. 2 " " " " " " " "



L = {w | w contains symbol 1 in 2nd last in right end}

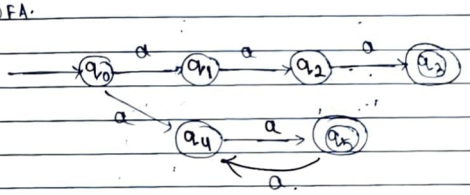
	0	1
q_0	q_0	$\{q_0, q_1\}$
q_1	q_2	q_2
q_2	ϕ	ϕ

	0	1
q_0	q_0	q_0q_1
q_0q_1	q_0q_2	$q_0q_1q_2$
q_0q_2	q_0	q_0q_1
$q_0q_1q_2$	q_0q_2	$q_0q_1q_2$



← DFA

Example 3 → convert the following NFA to its equivalent DFA.



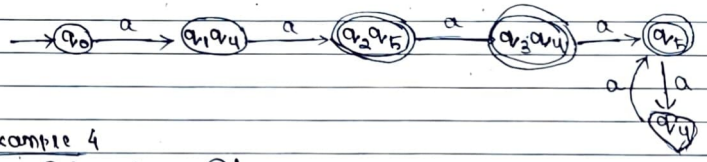
L = {w | w contains exactly 3 a's or even no. of a's except ε} over Σ = {a}

NFA

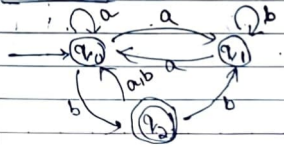
states / input	a
q_0	$\{q_1, q_4\}$
q_1	$\{q_2\}$
q_2	$\{q_3\}$
q_3	ϕ
q_4	$\{q_5\}$
q_5	$\{q_4\}$

DFA

	a
q_0	q_1q_4
q_1q_4	q_2q_5
q_2q_5	q_3q_4
q_3q_4	q_5
q_5	q_4
q_4	q_5



Example 4



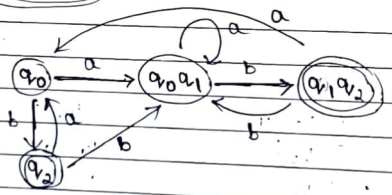
L = { w | ends with odd no. of b's over Σ = {a,b} }

state/input	a	b
q_0	$\{q_1, q_0\}$	$\{q_2\}$
q_1	$\{q_0\}$	$\{q_1\}$
q_2	$\{q_0\}$	$\{q_0, q_1\}$

NFA

DFA

	a	b
q_0	$q_0 q_1$	q_2
$q_0 q_1$	$q_0 q_1$	$q_1 q_2$
q_2	q_0	$q_0 q_1$
$q_1 q_2$	q_0	$q_0 q_1$



ε – NFA (Epsilon NFA):-

* ε represents empty symbol or null symbol.
* Every state on ε goes to itself

$\rightarrow (q_0)$ $\quad \delta(q_0, \varepsilon) = q_0$.

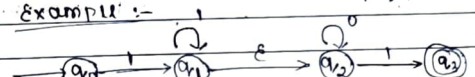
* The ε-NFA can also be defined like a regular NFA in 5-tuple form as $(Q, \Sigma, \delta, q_0, F)$. But ε-NFA differs only on transition mapping δ from regular N-FA

* In ε-NFA, the transition mapping δ is defined as δ maps from

$$\delta : Q \times \{\Sigma \cup \{\varepsilon\}\} \rightarrow 2^{Q}$$

* This means that the state on seeing an empty symbol could go to any state

Example :-



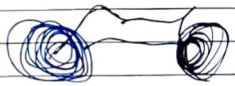
ε-NFA

state/input	0	1	ε
q_0	ϕ	q_1	q_0
q_1	ϕ	q_1	$\{q_1, q_2\}$
q_2	q_2	q_3	q_2
q_3	ϕ	ϕ	q_3

Example-2



	0	1	ε
q_0	q_0	ϕ	$\{q_0, q_2\}$
q_2	ϕ	$\{q_2, q_3\}$	$\{q_2, q_3\}$
q_3	ϕ	ϕ	$\{q_3\}$



Conversion of ε-NFA to NFA:

* Procedure:
  - → For each state, check where does this state go on ε* (Epsilon closure) then,
  - → check the states that we get here, on which state does they go on getting a particular input from Σ.
  - → Then the states that we get here have to be again checked on to which state to they go on ε* again.

* Defin of ε*:→ Epsilon closure means all the states that can be reached from a particular state only by seeing the epsilon symbol.

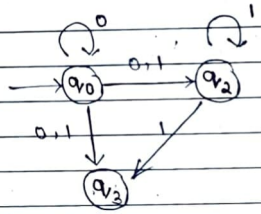
input / states	ε*	0	ε*
q_0	$\{q_0, q_2, q_3\}$		
q_2	$\{q_2, q_3\}$		
q_3	q_3		

q_0	q_0	q_0	$\{q_0, q_2, q_3\}$	
	q_2	ϕ	ϕ	
	q_3	ϕ	ϕ	
q_2	q_2	ϕ	ϕ	
	q_3	ϕ	ϕ	
q_3	q_3	ϕ	ϕ	

input / states	ε*	1	ε*
q_0	q_0	ϕ	ϕ
	q_2	$\{q_2, q_3\}$	$\{q_2, q_3\}$
	q_3	ϕ	ϕ
q_2	q_2	$\{q_2, q_3\}$	$\{q_2, q_3\}$
	q_3	ϕ	ϕ
q_3	q_3	ϕ	ϕ

create the transition table of normal NFA

states	0	1
→ q_0	$\{q_0, q_2, q_3\}$	$\{q_2, q_3\}$
q_2	ϕ	$\{q_2, q_3\}$
(q_3)	ϕ	ϕ



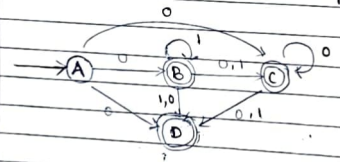
Q2

states	ε*	0	ε*
A	A	B	{B, C, D}
B	B	ϕ	ϕ
C	C	C	{C, D}
D	D	ϕ	ϕ
C	C	C	{C, D}
	D	ϕ	ϕ
D	D	ϕ	ϕ

States	ε^*	1	ε^*
A	A	ϕ	ϕ
B	B	B	{B, C, D}
	C	D	D
	D	ϕ	ϕ
C	C	D	D
	D	ϕ	ϕ
D	D	ϕ	ϕ

NFA

input / state	0	1
A	{B, C, D}	ϕ
B	{C, D}	{B, C, D}
C	{C, D}	D
D	ϕ	ϕ



A state will become final state, it in ε-NFA it can reach final state by seeing ε.

* Regular language (RL) :- A language is said to be RL if some finite automata recognizes/accepts it.

Regular Operations

The operators operated on the regular languages are called Regular operators.

1) Union (U)
2) concatenation (.)
3) star / Kleene star / Kleene closure / Kleen (*)

Example :-

let A = {a, b} and B = {x, y} are two regular langua

A ∪ B = {a, b, x, y}
= {x | x ∈ A or x ∈ B}

A.B ≅ AB = {ax, ay, bx, by}

A* = {ε, a, b, {aa}, {ab}, {ba}, {bb}, {aaa}, ——— }.

A+ ⇒ A* − {ε}.

Theorem :-

The class of regular languages is closed under union operation. That is if L1 & L2 are two regular language then L1 ∪ L2 is also a regular language.

pf: [use proof by construction].

since L1 & L2 are two regular languages, there exists some finite automata M1 & M2 to recognize L1 & L2 respectively.

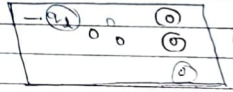
define M1 & M2 in 5-tuple form as:

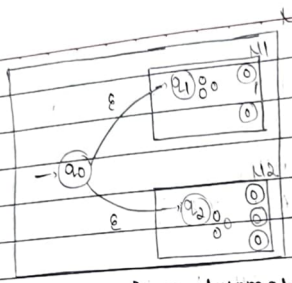
M1 = (Q1, Σ1, δ1, q1, F1) and
M2 = (Q2, Σ2, δ2, q2, F2).

let us construct M1 & M2 as:

M1







let N is the finite Automata that accepts $L_1 \cup L_2$

$N = (Q, \Sigma, \delta, q_0, F)$

↓ set of ~~state~~

where

1) $Q = \{q_0\} \cup Q_1 \cup Q_2$
2) $\Sigma = \Sigma_1 \cup \Sigma_2$
3) $q_0 \notin Q$ is the initial state
4) $F = F_1 \cup F_2$ $c_1 Q$ is set of accept/final state.
5) δ is the transition function Now define δ for any state $q \in Q$ & any input $a \in \Sigma$

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & \text{if } q \in Q_1 \\ \delta_2(q, a) & \text{if } q \in Q_2 \\ \{q_1, q_2\} & \text{if } q = q_0 \text{ \& } a = \varepsilon \\ \phi & \text{if } q = q_0 \text{ \& } a \neq \varepsilon \end{cases}$$

Theorem :- The class of regular languages is closed under concatenation operation that is, if L_1 & L_2 are two regular languages then $L_1 L_2$ is also a regular language

$L_1 \cdot L_2$ ✓

Pf: [use proof by construction]

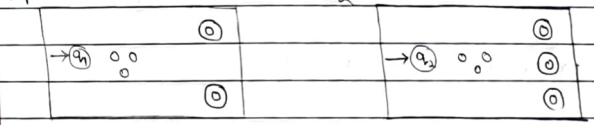
Let N_1 be the FA that accepts L_1 & N_2 be the FA that accepts L_2

Define N_1 & N_2 in 5-tuple form as

$N_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$ and

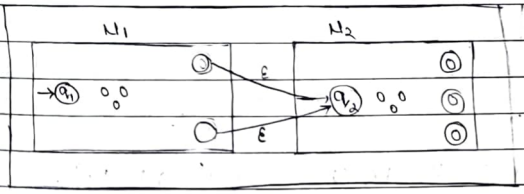
$N_2 = (Q_2, \Sigma_2, \delta_2, q_2, F_2)$

Now construct N_1 & N_2 as



let N be the FA that accepts $L_1 L_2$. so, $L_1 L_2$ is a regular language as it is accepted by N.

Now construct N as



Now define N in 5-tuple form as

$N = (Q, \Sigma, \delta, q_0, F)$ where

1) $Q = Q_1 \cup Q_2$
2) $\Sigma = \Sigma_1 \cup \Sigma_2$
3) $q_0 = q_1$ is initial state
4) $F = F_2$. $F \subseteq Q$ & the set of final states

5) δ is the transition function Now define δ for any state $q \in Q$ & any input $a \in \Sigma$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & \text{if } q \in Q_1 \text{ \& } q \notin F_1 \\ \delta_1(q,a) & \text{if } q \in F_1 \text{ \& } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & \text{if } q \in F_1 \text{ \& } a = \varepsilon \\ \delta_2(q_2,a) & \text{if } q \in Q_2 \end{cases}$$

Theorem The class of regular language is closed under star operation. That is, if L_1 is a regular language then L_1^* is a regular language.

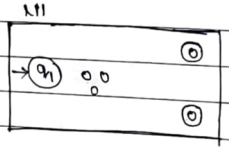
pf: [We proof by construction]

Let N_1 be FA that accept L_1.

Define N_1 in 5-tuple form as

$N_1 = (Q_1, \Sigma_1, \delta_1, q_1, F_1)$

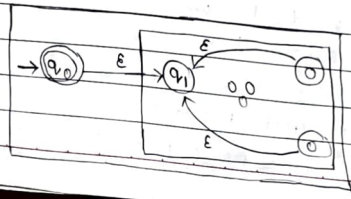
construct N_1 as:



Let N be the FA that will accept L_1^*.

This implies L_1^* is a regular language.

Define & Now construct N as:

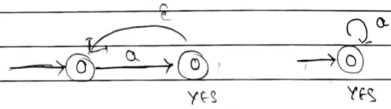


$N = (Q, \Sigma, \delta, q_0, F)$

where,

1) $Q = \{q_0\} \cup Q_1$
2) $\Sigma = \Sigma_1$
3) $q_0 \in Q$ is the initial state
4) $F = \{q_0\} \cup F_1$ is the set of final state.
5) δ is transition function. Define δ for any $q \in Q$ & any $a \in \Sigma$ as:

$$\delta(q,a) = \begin{cases} \delta_1(q,a), & \text{if } q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q,a), & \text{if } q \in F_1 \text{ and } a \neq \varepsilon. \\ \delta_1(q_1,a) \cup \{q_1\}; & \text{if } q \in F_1 \text{ and } a = \varepsilon. \\ \{q_1\} & \text{if } q = q_0 \text{ and } a = \varepsilon. \\ \phi & \text{if } q = q_0 \text{ \& } a \neq \varepsilon. \end{cases}$$



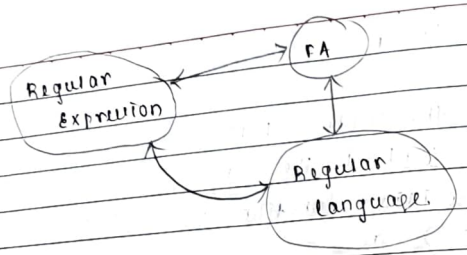
Regular Expressions :-

* There are the expressions built up by using regular operations

  e.g.

  - $(a \cup b)a^*$ is a regular expression
  - $(a \cup b) \cdot a^*$ ← closure operation
    - ↑ union operation
    - ↑ concatenation operation

* the regular expressions are used to describe the regular languages. i.e. the value of a regular expression is a regular language.



* There are useful tools in the design of compilers for programming languages. The elemental objects in a programming language called tokens such as the variable names, keywords, constants & punctuations are described with regular expression ↘ operators also.

certain rules for regular expressions

- Any terminal symbol belongs to the Alphabet Σ including ε and ϕ are regular expressions.

e.g. $\Sigma = \{a, b, c\}$ — Terminal symbol

- The union of two regular expression is a regular expression. That is, if R_1 & R_2 are two regular expressions then $R_1 \cup R_2 \approx R_1 + R_2$ is a regular expression

- The concatenation of two Regular expression is a regular expression i.e., if R_1 & R_2 are two regular expressions then $R_1 R_2$ is a regular expression

- The star of any regular expression is also a regular expression i.e. if R is a regular expression then R^* is also a Regular expression.

Examples on Regular expression :→

① consider the language as $L = \{a, b, c\}$,
The Regular Expression
$R = a \cup b \cup c$ or $a + b + c$

② consider the language as
$L = \{ab, a, abb, ac\}$

$R = ab \cup a \cup abb \cup ac \approx ab + a + abb + ac$

③ consider $L = \{\varepsilon, a\}$, then
$R = \{\varepsilon \cup a\}$

④ consider $L = \{\varepsilon, a, aa, aaa, \text{———}\}$
$R = a^*$

⑤ consider $L = \{a, aa, aaa, \text{———}\}$
$R = a^+$

⑥ consider $L = \{b, ab, aab, aaab, aaaab, \text{———}\}$
$R = a^*b$.

⑦ consider $L = \{\varepsilon, a, aa, aaa \text{———}, bbb\}$
$R = a^* \cup bbb \approx a^* + bbb$

⑧ consider $L = \{\}$
$R = \phi$

⑨ consider $L = \{\varepsilon\}$
$R = \varepsilon$.

10. consider $L = \{ab, abb, abbb, \ldots\}$
Then $R = ab^+$.

example

Determine the language when the regular expression is given:

1. let $\Sigma = \{0,1\}$ and
$R = (0+1)0^*$, then
the regular language
$L = \{w \in (0,1) \mid w$ starts with 0 or 1 followed by any number of 0's$\}$.

2. let $\Sigma = \{0,1\}$ and
$R = 0^*10^*10^*$ then
the language $L = \{w \in \{0,1\} \mid w$ contains any number of 0's & exactly two 1's$\}$.

3. let $\Sigma = \{0,1\}$ and
$R = (0+1)^* 00 (0+1)^*$ & the Language
$L = \{w \in \{0,1\} \mid w$ contains any number of 0's or 1's with atleast two consecutive 0's$\}$.
or
"00" as a substring

4. let $\Sigma = \{0,1\}$ &
$R = \Sigma^* 11 \Sigma^*$ then
$L = \{\}$,
same as 3. "11" as a substring

5. let $\Sigma = \{0,1\}$ and
$R = 1^*(01^+)^*$ then
$L = \{w \in \{0,1\} \mid w$ contains & any number of 0's and 1's with every 0 followed by atleast one 1's$\}$

EQUIVALENCE of FA & Regular expression:
(i.e.) conversion from regular expression to FA).

e.g

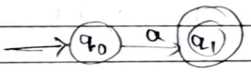
a) let $R = \phi$
$L = \{\}$



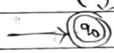
b) let $R = \varepsilon$
$L = \{\varepsilon\}$



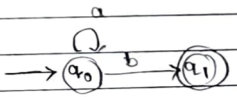
c) let $R = a$
where $\Sigma = \{a, b\}$



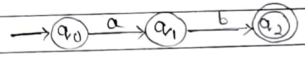
d) let $R = a^*$
$L = \{\varepsilon, a, aa, aaa, \ldots\}$



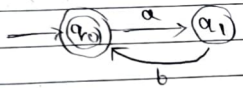
e) let $R = a^*b$
$L = \{b, ab, aab, \ldots\}$



f) Let $R = ab$
$L = \{ab\}$

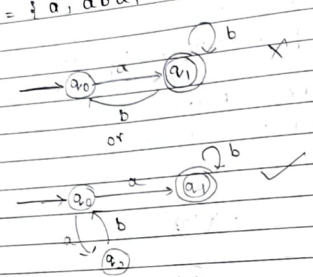


g) let $R = (ab)^*$
$L = \{\varepsilon, ab, abab, ababab, \ldots\}$



h) let R = (ab)* ab*

L = { a, aba, abab, ab, ... }

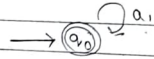


i) let R = a+b

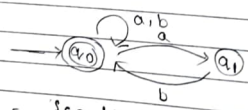
L = { a, b }



j) let R = (a+b)*



k) let R = (a+b)* (ab)*



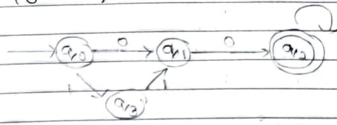


L_1 = { ε, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bbb, bab, ——— }

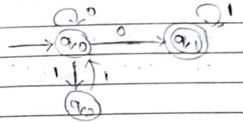
L_2 = { ε, ab, abab, ababab, ——— }

L = $L_1 . L_2$

l) let R = (0 + 11) 01*



m) let R = (0+11)* 01*



Identity rules for Regular expression :-

- $\phi + R = R$
- $\phi R + R\phi = \phi$
- $\varepsilon R = R\varepsilon = R$
- $\varepsilon^* = \varepsilon$ and $\phi^* = \varepsilon$.
- $R + R = R$
- $R^* R^* = R^*$
- $RR^* = R^* R = R^+$
- $(R^*)^* = R^*$
- $\varepsilon + RR^* = \varepsilon + R^* R = R^*$
- $(PQ)^* P = P(QP)^*$
- $(P + Q)^* = (P^* Q^*)^* = (P^* + Q^*)^*$
- $(P + Q)R = PR + QR$
- $R(P + Q) = RP + RQ$

e.g. 1

Prove the regular expression

$R = \varepsilon + 1^*(011)^*(1^*(011)^*)^*$ is equal to $(1 + 011)^*$

$R = \varepsilon + 1^*(011)^*(1^*(011)^*)^*$

using identity

$\varepsilon + RR^* = R^*$

$\Rightarrow (1^*(011)^*)^*$

$\Rightarrow (1^* + (011)^*)^*$

$\Rightarrow (1 + 011)^*$ By using identity $(P^*Q^*)^* = (P+Q)^*$

(proved)

e.g 2 Pf that $(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1) = 0^*1(0 + 10^*1)^*$

L.H.S $(1 + 00^*1) + (1 + 00^*1)(0 + 10^*1)^*(0 + 10^*1)$

$\Rightarrow (1 + 00^*1)[\varepsilon + (0 + 10^*1)^*(0 + 10^*1)]$

$\Rightarrow (1 + 00^*1)(0 + 10^*1)^*$

$\Rightarrow (\varepsilon + 00^*)1(0 + 10^*1)^*$

$\Rightarrow 0^*1(0 + 10^*1)^*$

proved.

Conversion from FA to regular expression:-

2 ways are there:

1. by using Arden's theorem
2. by using state-elimination method.

Arden's Theorem

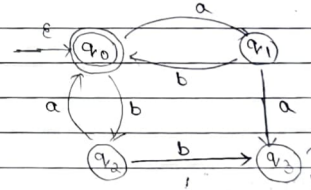
If P & Q are 2 regular expression over the alphabet Σ, and if P doesn't contain ε, then the equation $R = Q + RP$ has a unique solution $R = QP^*$ for some R.

e.g 1 Find the regular expression from the

procedure to find the Regular expression from the FA by using Arden's theorem

1) For each state find out the transition which comes from other state to that particular state. then,

2) Find out soln for the state which is in final state & leave other states as it is.



soln: Find the incoming transition for each state.

$q_0 = \varepsilon + q_1 b + q_2 a$ — ①

$q_1 = q_0 a$ — ②

$q_2 = q_0 b$ — ③

$q_3 = q_1 a + q_2 b + q_3 (a + b)$ — ④

Now solve equation ①,

$q_0 = \varepsilon + q_1 b + q_2 a$

$= \varepsilon + (q_0 a) b + (q_0 b) a$

$R = \varepsilon + q_0ab + q_0ba$

$\Rightarrow \underset{R}{q_0} = \underset{Q}{\varepsilon} + \underset{R}{q_0}\underset{P}{(ab+ba)}$

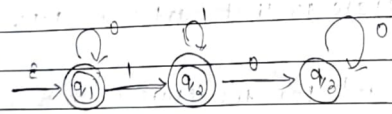
As it is satisfying Arden's theorem,

$\Rightarrow q_0 = \varepsilon(ab+ba)^*$

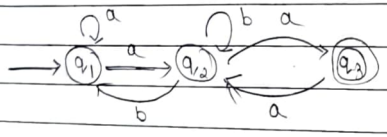
$= (ab+ba)^*$ is the required regular expression

Note: If 2 states are considered final then the union of these final state will give us regular expression.

e.g a)



b)



Ans a) $q_1 = \varepsilon + q_1 0$ *

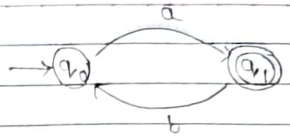
$q_2 = q_1 1 + q_2 1 = (q_1 + q_2)1$

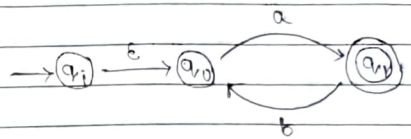
$q_3 = q_2 0 + q_3 0 + q_3 1 = (q_2 + q_3)0 +$

State Elimination method:-

Procedure :-

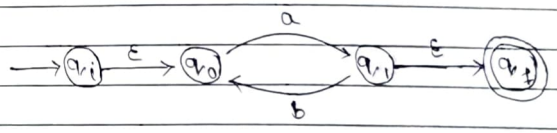
Step1 → If 7 any incoming transition to initial state then add a new initial state.



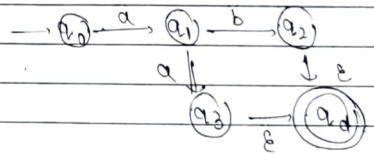


Step-2

- If 7 an outgoing edge to final state create a ne final state & add it to the Finite Automata.



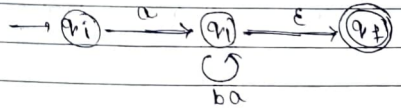
- If 7 multiple final states then convert these to Non-final states & create a single final state.



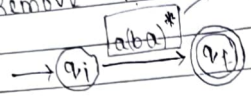
Step-3

- Eliminate all intermediate step one by one in any order.

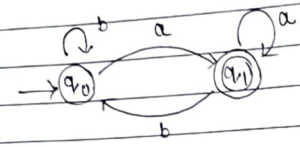
Remove or eliminate q_0.

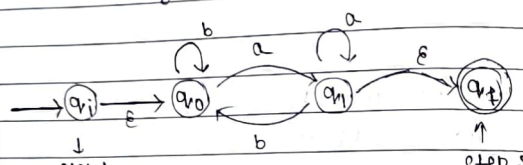


Remove q_1 → Regular expression

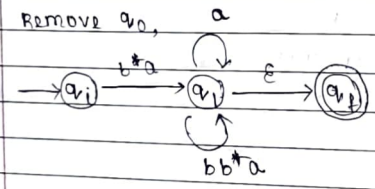


E.g 2

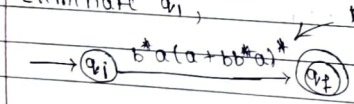




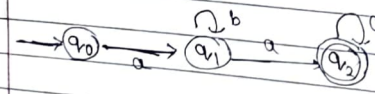
Step-3 Remove q_0,



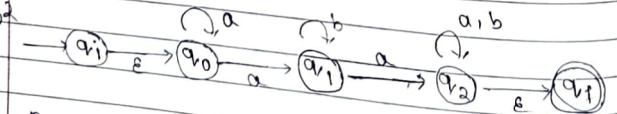
Eliminate q_1, → Regular expression



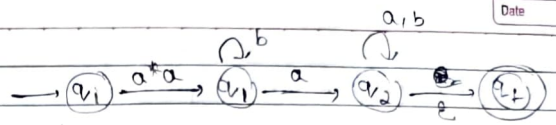
Ex.



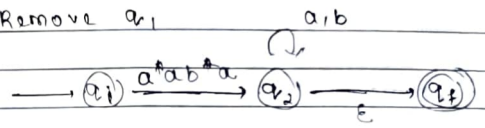
E-1,2



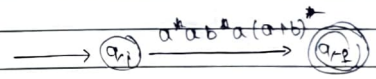
S-3 Remove q_0.



Remove q_1



Remove q_2



$$R = a^*ab^*a\,(a+b)^*$$

Non-Regular Language

A language which is not recognized or accepted by any finite automata is called a Non-regular language.

Pumping Lemma for Regular language

- It is a property that all regular languages must satisfy. that means if a language is regular then it must satisfy pumping lemma.
- If a language doesn't satisfy pumping lemma then it is called as a Non-Regular language.
- The pumping lemma is mainly used to prove that the language is not regular. It cannot be used to prove that a language is regular.

Q If L is a regular language then ∃ a constant $p > 0$ (called as pumping length) such that every string $w \in L$ with $|w| \geq p$, the string w can be divided into 3 parts as $w = xyz$, such that the following conditions must satisfy:

1) $|xy| \leq p$
2) $|y| > 0$
3) $xy^iz \in L$ for every $i \geq 0$.

e.g→ show that the language

$L = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Ans we proof by contradiction,

- let us consider L is regular
- ∃ a pumping length p i.e $p > 0$.
- $w = 0^p 1^p \in L$ where $|w| \geq 2p > p$
- let $p = 7$, $w = 0^7 1^7 = 00000001111111$
- now divide w into 3 parts:

$x = 000$
$y = 0000$
$z = 1111111$

As $|xy| \leq p$.
$|y| > 0$.
It satisfy pumping lemma cond^n ① & ②

③ $w = xy^iz$

For $i = 0$,

$w = xy^0z$
$= xz$
$= 0001111111 = 0^3 1^7 \notin L$

⇒ The 3rd cond^n is unsatisfied.
⇒ This contradict our assumption thus, L is not regular.

e.g→ prove that the language

$L = \{yy \mid y \in \{0,1\}^*\}$ is not regular.

A proof by contradiction,

- Assume L be regular
- As per pumping lemma, ∃ a pumping length $p \leq 1$ $p > 0$
- let $w = 0^p 1 0^p 1$, where $|w| = 2p + 2 > p$
- let $p = 3$, $w = 0^3 1 0^3 1$
  $= 00010001$
- now divide w into 3 parts

$x: 0$
$y: 00$
$z: 10001$

i) $|xy| \leq p$ ✓
ii) $|y| > 0$ ✓
iii) $w = xy^iz \notin L$ for $i \geq 0$

③ For $i = 0$

$w = xy^iz$
$= xz$
$= 010001 \notin L$

⇒ 3rd cond^n not satisfied
⇒ our assumption is contradicting.
∴ so, L is not regular.

Equivalence of two finite automata:

Steps (M1, M2)

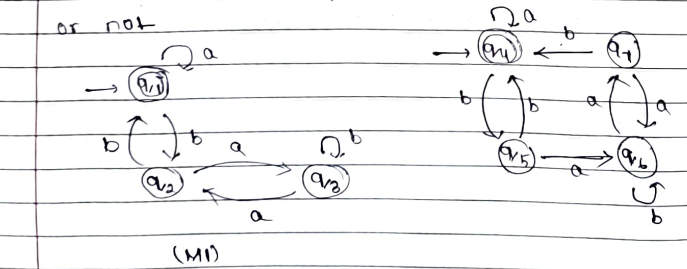
1. For any pair of states $\{q_i, q_j\}$, the transition for input $a \in \Sigma$ is defined by $\{q_{ia}, q_{ja}\}$ where

$$\delta(q_i, a) = q_{ia}$$
$$\text{and } \delta(q_j, a) = q_{ja}$$

The two automata are not equivalent if for a pair $\{q_{ia}, q_{ja}\}$ one is intermediate state & other is final state.

2. If initial state is final state of one automata then in second automaton also initial state must be final state for them to be equivalent.

e.g. check the following two FA are equivalent or not

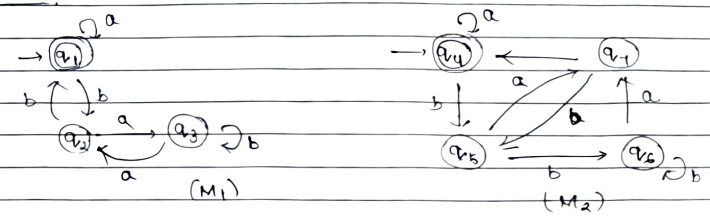


(M1)

pair of states	a	b	
$\{q_1, q_4\}$	$\{q_1, q_4\}$	$\{q_2, q_5\}$	✓
$\{q_2, q_5\}$	$\{q_3, q_6\}$	$\{q_1, q_4\}$	✓
$\{q_3, q_6\}$	$\{q_2, q_7\}$	$\{q_3, q_6\}$	✓
$\{q_2, q_7\}$	$\{q_3, q_6\}$	$\{q_3, q_4\}$	✓

so, M_1 & M_2 are equivalent.

Ex. 2



(M_1) (M_2)

pair of states	a	b	ε
$\{q_1, q_4\}$	$\{q_1, q_4\}$	$\{q_2, q_5\}$	✓
$\{q_2, q_5\}$	$\{q_3, q_7\}$	$\{q_1, q_6\}$	X

(final → intermediate)

Both M_1 & M_2 are not equivalent.

Q Minimal DFA

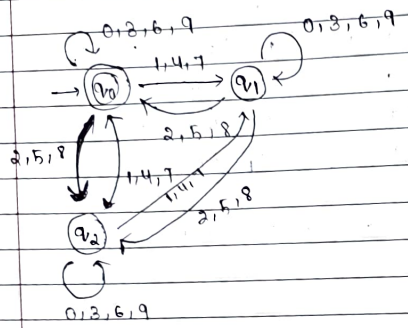
$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

$w \in \Sigma^*$

sum(w) → (sum of digits/symbols in w) modulo 3.

0 mod 3 = 0	(0, 3, 6, 9) → 0
1 " " = 1	(1, 4, 7) → 1
2 " " = 2	(2, 5, 8) → 2.

$L = \{w \in \Sigma^* \mid sum(w) = 0\}$



CH-2

CONTEXT-FREE LANGUAGES (CFL)

* collection of language associated with context-free grammar are called (CFL). They include all regular languages & many additional languages.

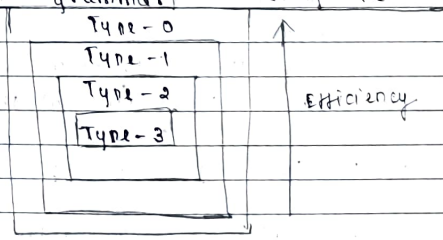
[1] chomsky Hierarchy

* Noam chomsky gave a mathematical model of grammar which is effective for writing computer languages.

* there are 4 types of grammar acc. to Noam chomsky

Grammar

Type	Accepted	language	Automata
Type-0	Unrestricted grammar	Recursively enumerable language.	Turing machine
Type-1	context-sensitive grammar	context sensitive language	Linear-Bounded automata
Type-2	context-free grammar	context free language	push-down automata
Type-3	Regular grammar	Regular language	Finite Automata



CONTEXT FREE GRAMMAR :-

* It consists of a collection of substitution rules called production rules.

* Each rule appears as a line in the grammar comprising a symbol & a string separated by an Arrow.

A symbol ⟶ A string

e.g $A \longrightarrow aAb$

* The symbol is called a variable or non-terminal symbol. The string consists of variables & other symbols called terminals

↓ upper-case ↗ L, input Alphabet(Σ) → lower-case

* The terminals are represented by letters, numbers, special symbols.

* one variable is designated as the start variable of the grammar.

It usually occurs on the left-hand side of the topmost / First rule of the grammar.

e.g → The following is an e.g of grammar

$A \longrightarrow 0A1$
$A \longrightarrow B$
$B \longrightarrow \#$

variables = {A, B}
terminals = {0, 1, #}
start variable ⟶ A.

FORMAL DEFINITION OF CONTEXT-FREE GRAMMAR

☐ It is a 4-tuple (V, Σ, P, S) where

i) V is a finite set of variables / non-terminals.

ii) Σ is a finite set of terminals.

iii) P is a finite set of production rules in the form

$A \rightarrow \alpha$, where

$A \in V$ and

$\alpha \in (V \cup \Sigma)^*$

iv) $S \in V$ is the start variable of the grammar.

we can write

(V, T, P, S)
↓
(V, T, R, S) } Both are same
↓
Σ

e.g → consider the following context-free grammar G with its production rules as

$A \longrightarrow 0A1$
$A \longrightarrow B$
$B \longrightarrow \#$

Ans $V = \{A, B\}$
$\Sigma = \{0, 1, \#\}$
$S = A$

e.g
$X \rightarrow A$
$X \rightarrow B$ } ⟶ $X \rightarrow A \mid B \mid C$.
$X \rightarrow C$

4-tuple $(V, \Sigma, P, S) = (\{A, B\}, \{0, 1, \#\}, \{A \rightarrow 0A1 \mid B, B \rightarrow \#\}, A)$.

□ Derivation of strings from a grammar

Eg- Derive the string "000#111" from the grammar:

{ A → 0A1 | B, B → # }

* A → 0A1

↓

00A11 [∵ A → 0A1]

↓

000A111 [∵ A → 0A1]

↓

000B111 [∵ A → B]

↓

[∵ B → #] 000#111 (Derived)

2) A ⟶ B

⇒ # [∵ B → #]

3) A → 0A1

⇒ 0B1 [∵ A → B]

⇒ 0#1 [∵ B → #]

4) A → 0A1

↓

00A11 [∵ A → 0A1]

↓

00B11 [∵ A → B]

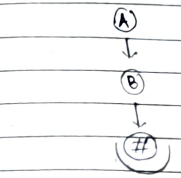
↓

00#11 [∵ B → #]

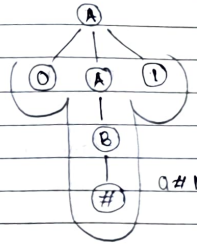
$L = \{0^n \# 1^n \mid n \geq 0\}$

DERIVATION TREE / PARSE TREE.

2) A → B → #

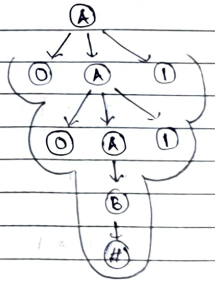


3) A → 0A1
→ 0B1
→ 0#1



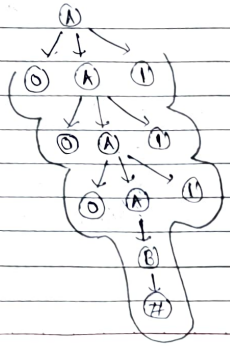
0#1

4) A → 0A1
→ 00#11



00#11

1) A → 0A1
→ 000#111



PETALS Page No. Date / /

[1] There are 2 types of Derivation tree:

1) left derivation tree (LDT)
2) Right Derivation Tree (RDT)

1) A LDT is obtained by applying production rules to the leftmost variable in each step.

2) " RDT " " " " " "
" " " Rightmost " " " " .

e.g- consider the grammar G with its production rules as:

$S \rightarrow S+S$
$S \rightarrow S*S$
$S \rightarrow a$
$S \rightarrow b$

$S \longrightarrow S+S \mid S*S \mid a \mid b$

* create a left derivation tree

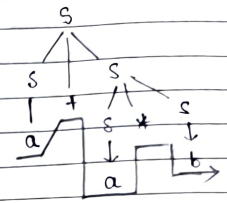
Ans e.g- Derive string $(a+a)*b$
or
"$a+a*b$"

$S \rightarrow S+S$
$\rightarrow a+S$ [$\because S \rightarrow a$]
$\rightarrow a+S*S$ [$\because S \rightarrow S*S$]
$\rightarrow a+a*S$ [$\because S \rightarrow a$]
$\rightarrow a+a*b$ [$\because S \rightarrow b$]

NOTEBOOK

PETALS Page No. Date / /

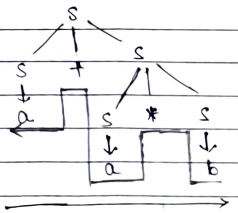
(L.D.T)



Q create a Right Derivation Tree.

$S \rightarrow S+S$
$\rightarrow S+S*S$ [$\because S \rightarrow S*S$]
$\rightarrow S+S*b$ [$\because S \rightarrow b$]
$\rightarrow S+a*b$ [$\because S \rightarrow a$]
$\rightarrow a+a*b$. [$\because S \rightarrow a$]

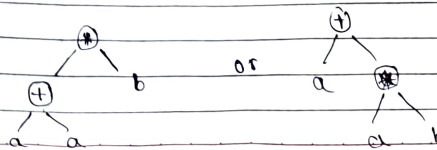
(R.D.T)



AMBIGUOUS GRAMMAR

[1] A grammar is said to be Ambiguous if ∃ two or more L.D.T or two or more R.D.T

e.g $a+a*b$



$S \rightarrow S*S$

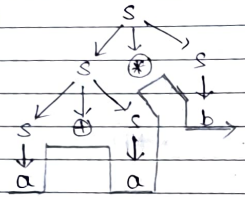
$\rightarrow S+S*S$ [$\because S \rightarrow S+S$]

$\rightarrow a+S*S$ [$\because S \rightarrow a$]

$\rightarrow a+a*S$ [$\because S \rightarrow a$]

$\rightarrow a+a*b$ [$\because S \rightarrow b$]

B.D.T



a+a*b.

$S \rightarrow S*S$

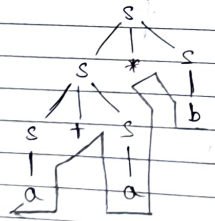
$\rightarrow S*b$ [$\because S \rightarrow b$]

$\rightarrow S+S*b$ [$\because S \rightarrow S+S$]

$\rightarrow S+a*b$ [$\because S \rightarrow a$]

$\rightarrow a+a*b$ [$\because S \rightarrow a$]

B.D.T



a+a*b.

so, Ambiguous Grammar.

for the e.g.

e.g 2 check the following grammar is ambiguous or not for the string "5*2+3".

$E \rightarrow I$

$E \rightarrow E+E$

$E \rightarrow E*E$

$E \rightarrow (E)$

$I \rightarrow \varepsilon \mid 0 \mid 1 \mid 2 \mid \cdots \mid 9$

L.D.T

(1) $E \rightarrow E+E$

$\rightarrow E*E+E$ [$\because E \rightarrow E*E$]

$\rightarrow I*E+E$ [$\because E \rightarrow I$]

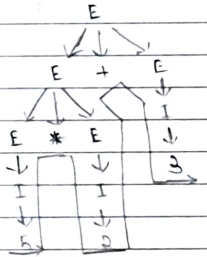
$\rightarrow 5*E+E$ [$\because I \rightarrow 5$]

$\rightarrow 5*I+E$ [$\because E \rightarrow I$]

$\rightarrow 5*2+E$ [$\because I \rightarrow 2$]

$\rightarrow 5*2+I$ [$\because E \rightarrow I$]

$\rightarrow 5*2+3$ [$\because I \rightarrow 3$]



(2) $E \rightarrow E*E$

$\rightarrow I*E$ [$\because E \rightarrow I$]

$\rightarrow 5*E$ [$\because I \rightarrow 5$]

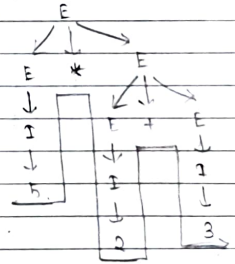
$\rightarrow 5*E+E$ [$\because E \rightarrow E+E$]

$\rightarrow 5*I+E$ [$\because E \rightarrow I$]

$\rightarrow 5*2+E$ [$\because I \rightarrow 2$]

$\rightarrow 5*2+I$ [$\because E \rightarrow I$]

$\rightarrow 5*2+3$ [$\because I \rightarrow 3$]



so, this grammar is Ambiguous.

Do same for RDT it will also show Ambiguowity.

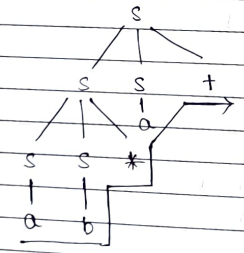
e.g-2 check the following grammar is Ambiguous or not for string "5*2+3"

$S \rightarrow SS+ \mid SS* \mid a \mid b$

5 2 * 3 +
"ab*a+"

L.D.T

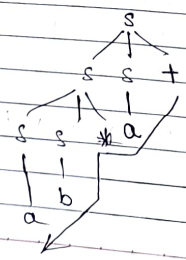
$S \rightarrow SS*$
$\rightarrow SS*S+$ [∵ $S \rightarrow SS*$]
$\rightarrow aS*S+$ [∵ $S \rightarrow a$]
$\rightarrow ab*S+$ [∵ $S \rightarrow b$]
$\rightarrow ab*a+$ [∵ $S \rightarrow a$]



Not Ambiguous Grammar

R.D.T

$S \rightarrow SS+$
$\rightarrow SS*S+$
$\rightarrow SS*a+$
$\rightarrow Sb*a+$
$\rightarrow ab*a+$
$\rightarrow SS*a+$
$\rightarrow Sb*a+$
$\rightarrow ab*a+$



CHOMSKY NORMAL FORM (CNF)

* A context free grammar is in chomsky Normal Form if the production rules are in the following forms.

$A \rightarrow a$
$A \rightarrow BC$

where A, B, C are variables & a is the terminal symbol.

That means in CNF, there is restriction on the length of RHS of any production rule, which is, the elements in RHS should either be two variables or a single terminal.

* procedure to convert a CFG to CNF:

step-1: If start symbol S occurs on the RHS of some production rule then create a new start symbol S' & add a new production $S' \rightarrow S$ into the grammar

step-2: Remove all NULL Productions

step-3: Remove all unit productions.

step-4: Replace each production

$A \rightarrow B_1 B_2 \cdots B_n$ where $n > 2$ with the productions

$A \rightarrow B_1 C$ & $C \rightarrow B_2 B_3 \cdots B_n$

e.g- $A \rightarrow XYZP$

W

$A \rightarrow XW$

$W \rightarrow YZP$

$W \rightarrow YR$

$R \rightarrow ZP.$

$B \rightarrow bB \mid b$

$C \rightarrow c$

Removal of UNIT production

* Any production rule of form $A \rightarrow B$ is called a unit production A, B are variables.

Procedure to remove unit production

S-1

To remove production $A \rightarrow B$ add a production $A \rightarrow X$ to grammar whenever the production $B \rightarrow X$ occurs in the grammar

S-2

Delete the unit production $A \rightarrow B$ from grammar.

S-3

Repeat from S-1 until all unit productions are removed.

e.g.

$S \rightarrow Xy$
$X \rightarrow a$
$Y \rightarrow Z \mid b$
$M \rightarrow N$
$Z \rightarrow M$
$N \rightarrow a$

* Remove $M \rightarrow N$
$M \rightarrow N$
$N \rightarrow a$
New production
$M \rightarrow a$
* Remove $Z \rightarrow M$
$Z \rightarrow M$
$M \rightarrow a$
New production
$Z \rightarrow a$

* Remove $Y \rightarrow Z$
$Y \rightarrow Z$
$Z \rightarrow a$
New production
$Y \rightarrow a$

Final Grammar
$S \rightarrow XY$
$X \rightarrow a$
$Y \rightarrow a \mid b$
$Z \rightarrow a$
$M \rightarrow a$
$N \rightarrow a$

Conversion from CFG to CNF :-

e.g. convert the following CFG to CNF,

$S \rightarrow a \mid aA \mid B$
$A \rightarrow aBB \mid \varepsilon$
$B \rightarrow Aa \mid b$

Ans S1 - Start variable S is not present in any production

S2 - Remove $A \rightarrow \varepsilon$

$S \rightarrow a \mid aA \mid B$
$\downarrow \varepsilon$
$\rightarrow a \mid aA \mid B \mid a$
$\rightarrow a \mid aA \mid B$

$B \rightarrow Aa \mid b$
$\rightarrow Aa \mid b \mid a$

Grammar after removing $A \rightarrow \varepsilon$.

$S \rightarrow a \mid aA \mid B$
$A \rightarrow aBB$
$B \rightarrow Aa \mid b \mid a$

S-3 Remove unit production

$S \rightarrow B$

$S \rightarrow a \mid aA \mid Aa \mid bla$

New Grammar:

$S \rightarrow a \mid aA \mid Aa \mid b$

$A \rightarrow aBB$

$B \rightarrow Aa \mid b \mid a$

S-4

$S \rightarrow a \mid aA \mid Aa \mid b$

$A \rightarrow aX$

$X \rightarrow BB$

$B \rightarrow Aa \mid b \mid a$

S-5

Final Grammar:

$S \rightarrow a \mid YA \mid AY \mid b$

$A \rightarrow YX$

$B \rightarrow AY \mid b \mid a$

$X \rightarrow BB$

$Y \rightarrow a$

Grammar is in CNF.

e.g ② conversion from CFG to CNF

convert the following from CFG to CNF where the production rules are:

$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$

$B \rightarrow b \mid \varepsilon.$

Soln Rule No. 1.

S-1

$S^1 \rightarrow S$

$S \rightarrow ASA \mid aB$

$A \rightarrow B \mid S$

$B \rightarrow B \mid \varepsilon.$

S-2 Remove NULL productions $B \rightarrow \varepsilon$.

$S \rightarrow ASA \mid aB \mid a$

$A \rightarrow B \mid S \mid \varepsilon$

Grammar after removal of production $B \rightarrow \varepsilon$

$S^1 \rightarrow S$

$S \rightarrow ASA \mid aB \mid a$

$A \rightarrow B \mid S \mid \varepsilon$

$B \rightarrow b$

* Remove $A \rightarrow \varepsilon$.

$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$

New Grammar

$S^1 \rightarrow S$

$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS \mid S$

$A \rightarrow B \mid S$

$B \rightarrow b$

Step-3 Remove unit productions:

$S^1 \rightarrow S$, $S \rightarrow S$, $A \rightarrow B$, $A \rightarrow S$

① $S \rightarrow S$

$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS.$

② $S' \rightarrow S$

$S' \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

③ $A \rightarrow B$ & $A \rightarrow S$

$A \rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS$

New Grammar

$S' \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

$S \rightarrow ASA \mid aB \mid a \mid SA \mid AS$

$A \rightarrow b \mid ASA \mid aB \mid a \mid SA \mid AS$

$B \rightarrow b$

S-4 Normalize production whose right hand side contains more than 2 symbols

$A \rightarrow B_1 B_2 \text{ —— } B_n \quad n > 2$

$A \rightarrow B_1 C$

where $C = B_2 \text{ —— } B_n$

$S' \rightarrow AX \mid aB \mid a \mid SA \mid AS$

$S \rightarrow AX \mid aB \mid a \mid SA \mid AS$

$A \rightarrow b \mid AX \mid aB \mid a \mid SA \mid AS$

$B \rightarrow b$

$X \rightarrow SA$

S-5 Replace $S' \rightarrow aB$

$S \rightarrow aB$

$A \rightarrow aB$

New Grammar

$S' \rightarrow AX \mid YB \mid a \mid SA \mid AS$

$S \rightarrow AX \mid YB \mid a \mid SA \mid AS$

$A \rightarrow b \mid AX \mid YB \mid a \mid SA \mid AS$

$B \rightarrow b$

$X \rightarrow SA$

$Y \rightarrow a$

Pumping lemma for context free (CFL) language :→

If A is a context-free language then A has a pumping length p s.t. any string 's' belongs to A, and $|s| \geq p$ (where) we can divided into 5 parts, $s = uvxyz$ s.t. the following condⁿ must be true.

i) $|vy| > 0$

ii) $|vxy| \leq p$

iii) $uv^i xy^i z \in A$ for every $i \geq 0$.

The pumping lemma for CFL is used to prove that a language is not CFL.

Q Show that the language $L = \{ a^n b^n c^n \mid n \geq 0 \}$ is not context free language using pumping lemma.

Pf:

* let us assume L is context free-language.

∃ a pumping length p

let $s \in L$ where $|s| \geq p$

Assume $s = a^p b^p c^p$

let $p = 4$,

$s = aaaabbbbcccc$

divide s into 5 parts

$u = a$

$v = aa$

$x = a$

$y = b$

$z = bbbcccc$

① $|vy| = 3 > 0$ ✓

② $|vxy| = 4 = p$ ✓

PETALS | Page No.
Date / /

③ Let i=0

$w = uv^0xy^0z$

$= uxz$

$= aabbbcccc$

$= a^2b^3c^4 \notin L$

(NOT satisfied)

This contradict our assumption. So, L is not a context-free Language.

e.g $L = \{ww \mid w \in \{0,1\}^*\}$

so, $w = (0+1)^n (0+1)^n \quad n \geq 0$

let it be $L = 0^n 1 0^n 1$

let p be pumping length

$L = 0^p 1 0^p 1$

let p = 4

$L = 000010000 1$

Divide it

u → 0

v → 00

x → 0

y → 1

z = 0000 1

① $|vy| > 0$

② $|vxy| \leq p$

≤ 4

③ $L = uv^i xy^i z$

i=0, → uxz

→ 0000 1

$= 0^6 1 \notin L$

* contradict our Accumption

* So, NOT CFL.

PETALS | Page No.
Date / /

PUSHDOWN AUTOMATA (PA) (PDA)

(PDA)

* A (PA) is a new type of computational problem (PDA) model. It has an extra component called a stack. The stack provides additional memory to the pushdown automata is a combination of Finite state machine or Finite automata & a stack.

PDA = FSM + a stack

Top → C
B
A

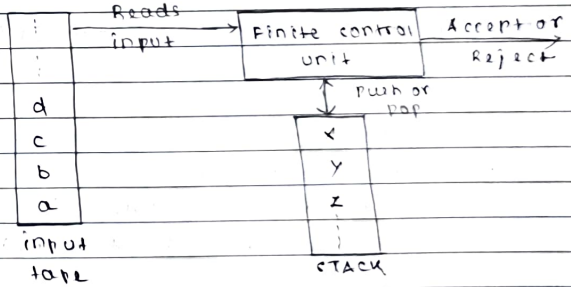
STACK

push (insertion)
pop (Deletion)

* It is more powerful than Finite Automata. A PDA has 3 components:-

i) An input tape (i.e input string)
ii) A finite control unit
iii) A stack with finite size.

Schematic Representation of PDA



Formal Definition of PDA

A PDA is formally defined by 7-tuples as:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$ where,

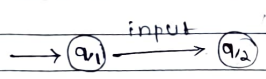
1) Q is finite set of states
2) Σ is the finite set of input symbols.
3) Γ is the (TaO) finite set of stack symbol
4) δ is transition function, defined as

$$\delta : Q \times \Sigma \times \Gamma \longrightarrow P(Q \times \Gamma) \simeq 2^{(Q \times \Gamma)}$$

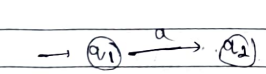
5) $q_0 \in Q$ is initial state.
6) Z_0 is the stack's finite symbol, used as a marker symbol.
7) $F \subseteq Q$ is the set of final states / accept states.

Design of a PDA

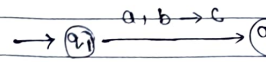
In a FA a transition from a state q_1 to q_2 is represented as:



e.g.



In PDA the transition from a state q_1 to another state q_2 can be represented as:

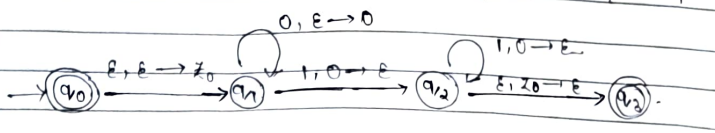


Here, a is input symbol
b is symbol to be popped from the stack,
c is symbol to be pushed onto the stack.

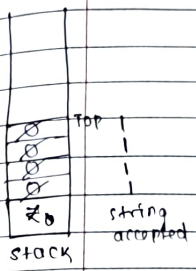
$$\delta : Q \times \Sigma \times \Gamma \longrightarrow P(Q \times \Gamma)$$

e.g-1

Design the PDA that accepts the language $L = \{0^n 1^n \mid n \geq 1\}$

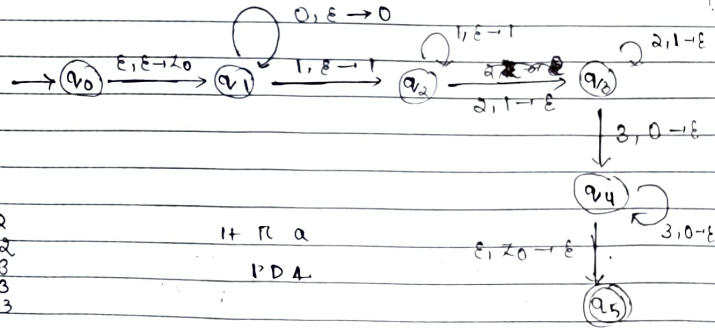


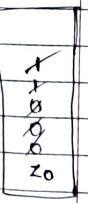
ω = 00001111 ✓
ω = 000011111 ✗



e.g-2:

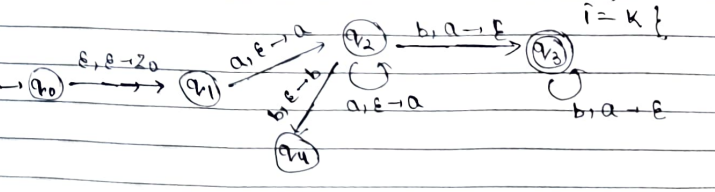
Design the PDA that accept language $L = \{0^n 1^m 2^m 3^n \mid n \geq 1 \text{ & } m \geq 1\}$

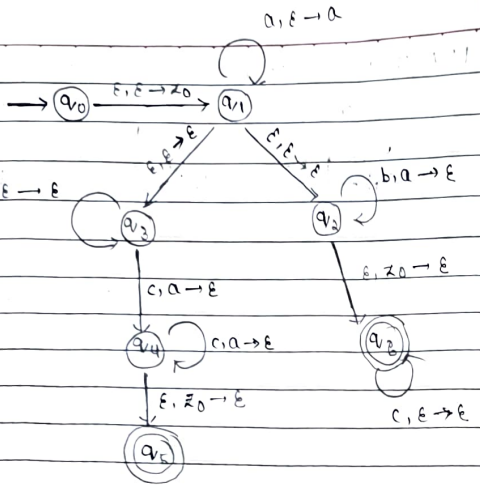




It is a PDA

$L = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i = j \text{ or } i = k\}$



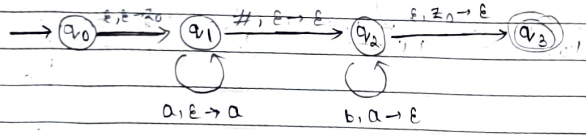


e.g - 4

Design a PDA that accepts the language

$L = \{ a^n \# b^n \mid n \geq 0 \}$.

Ans

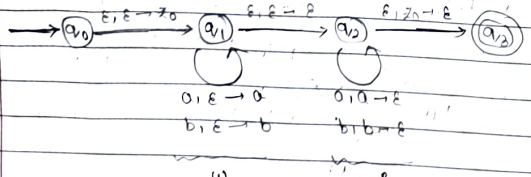


e.g → 5

Design a PDA that accepts even palindrome.

accepts $L = \{ ww^R \mid w \in \{a, b\}^* \}$

even length.



EQUIVALENCE WITH CFG

(conversion from CFG to PDA)

s-1 :- Push the marker symbol Z_0 & the start variable S of the context free grammar onto the stack.

s-2 :- Repeat following steps :

a) if the top of stack is a variable symbol A, non-deterministically select one of the rules for A & substitute A by the string on RHS of the rule.

a
b
c
S
Z_0

stack

$S \rightarrow abc$

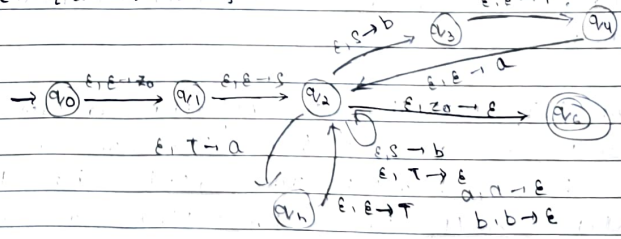
from right we will push.

b) if top of stack is a terminal symbol a, pop a from top of the stack if it matches with the input symbol & move forward to next symbol.

e.g - 1 construct PDA of given CFG. $S \rightarrow aTb \mid b$

$T \rightarrow Ta \mid \varepsilon$.

$L = \{ a^n b \mid n \geq 0 \}$



S

Z_0

TURING MACHINE

* Alan Turing in 1936, first proposed a much more powerful computational model called as Turing Machine.

* TM is similar to finite Automaton but with an unlimited and unrestricted (tape length/memory).

* A TM is a much more accurate model of a General purpose computer.

* A TM can do everything that a real computer can do.

CHURCH-TURING THESIS (CTT)

* Defⁿ came in the year 1936, the research paper of Alonzo Church & Alan Turing.

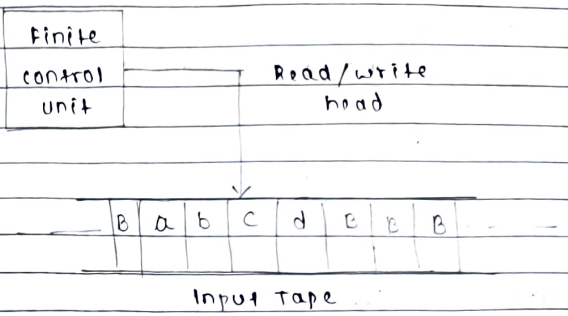
* church used a notational symbol/system called lambda calculus, to define Algorithms.

* But Turing did it with his machine. These two defⁿ where shown to be equivalent.

* The connection between these informal notion of Algo. and precise definition is called church Turing Thesis.

* The CTT state that any Algorithmic procedures that can be carried out by human beings or computers can be carried out by Turing Machine. i.e., Informal Notion of Algorithms = Turing Machine Algorithms.

Schematic Representation of TM :-



* The TM use infinite tape as its unlimited memory. It has a Tape head that can read & write symbols & move around the Tape.

* Initially the tape contains only the input string & is blank everywhere [␣ or B]

* If Machine needs to store info. It may write the info on tape & if the machine ~~needs to read info~~ needs the info that was written on the tape and machine can move its read/write head to left or to right.

Formal Defⁿ of TM :-

A Turing Machine is defined as a 7-tuple.

$(Q, \Sigma, \Gamma, \delta, q_0, B, F)$, where,

i) Q is a finite set of states.
ii) Σ is a finite set of input symbols.
iii) Γ (need as Tape) is the finite set of symbols.
iv) δ Transition funcⁿ, defined as:

$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$.

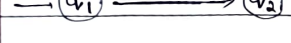
v) $q_0 \in Q$ is the initial state

vi) B is the blank

vii) $F \subseteq Q$ is the set of accept / reject state

Design of a TM :→

* A Transition from one state to other state can be represented as



Here, a is the symbol to read from tape

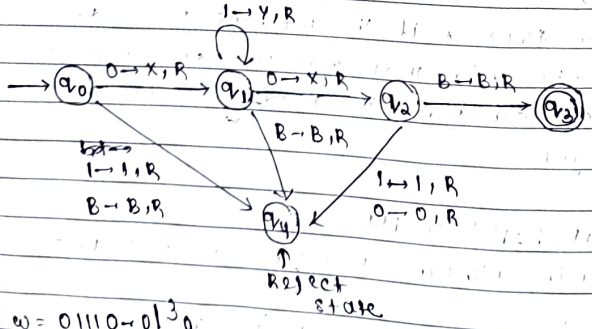
b is the symbol written on to the tape.

D is the director (L/R) to move the tape head.

e.g-①

Design a TM which recognizes the language

$L = \{0 1^n 0 \mid n \geq 0\}$



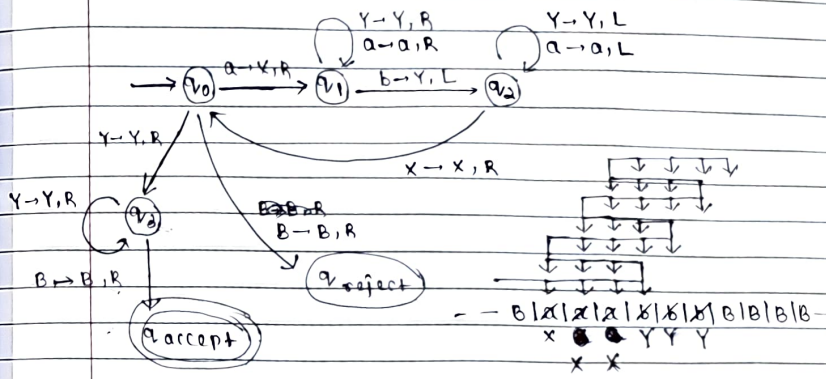
w = 01110→01 }0.



e.g ②

Design TM that recognizes the language

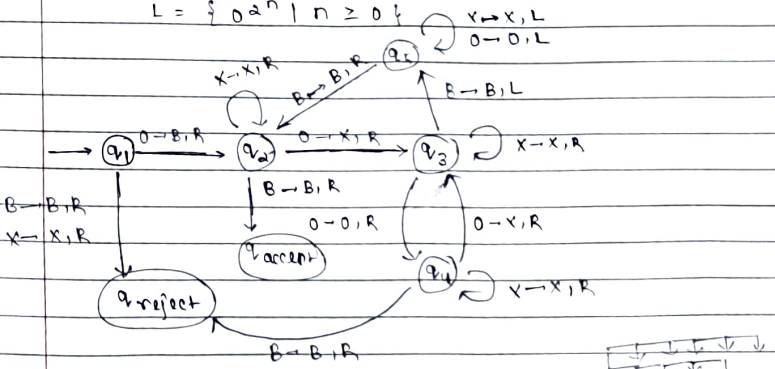
$L = \{a^n b^n \mid n \geq 1\}$

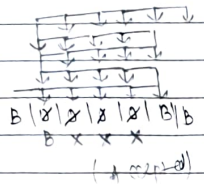


eg ③

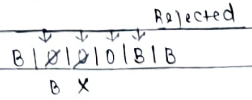
Design a TM that recognizes the language.

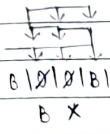
$L = \{0 a^n \mid n \geq 0\}$





Rejected





Accepted

e.g. 5

$L = \{ w \# w \mid w \in \{0,1\}^* \}$

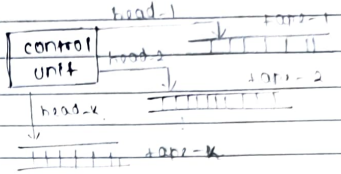
Design a TM for the language.

$w = 0101$

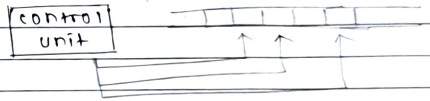
$L = 0101\#0101$

Ex: Variants of TM

1) Multi-Tape TM :→ It is like an ordinary TM with several tapes, each tape has its own head for reading & writing.



2) Multi-Head TM :- It is also like an ordinary TM with some fixed no. of heads. A move of TM depends on the state & the symbol scanned by each head.



3) Fixed Head TM : Here, the Tape head is fixed to a particular cell of the tape.



4) Non-Deterministic TM :→ In NDTM, at any point in a computation the machine may proceed acc. to several possibilities.

transition funcn

$\delta : Q \times \Gamma \to P(Q \times \Gamma \times \{L, R\})$

PETALS | Page No.
Date / /

Languages:

1) Recursive language :→ A language L is said to be recursive if ∃ a TM which will accept all the strings in L & reject all the strings that are not in L.

The TM will halt everytime & give an answer accepted rejected. for each & every string input.

2) Recursively enumerable language :→

A language L is said to be REL if ∃ a TM which will accept & therefore halt for all the input strings which are in L. But may or may not halt for all input strings which are not in L.

3) Decidable language :-
A language L is decidable, if it is a recursive language.

All decidable language are recursive languages & vice-versa.

4) Partially decidable language :- A language L is PD if it is recursively enumerable language.

All PDL are REL & vice-versa.

5) Undecidable language :- A language is Undecidable if it is not decidable or recursive language.

It may sometimes be partially decidable but not decidable.

PETALS | Page No.
Date / /

if a language is not PD then ∄ no turing machine for that language.

Halting problem : It is stated as " Given a program, will it halt ? ".

It says that can we design a machine in such a way that if we give a program to that machine as an input then that machine should be able to tell whether the program will halt or not halt on a particular input to that program.

H(P, I)

↳ undecidable problem.